

并行与分布式计算

-- 一致性与副本

性能和可扩展性

主要问题

为了保持副本的一致性，我们通常需要确保所有冲突的操作都以相同的顺序完成

冲突操作：从事务的角度来看

- ▶ **Read-write conflict** 读写冲突：一个读取操作和一个写入操作同时进行
- ▶ **Write-write conflict** 写入冲突：两个并发写入操作

问题

保证冲突操作的全局排序可能是一个代价高昂的操作，会降低可扩展性

解决方案：削弱一致性要求，以便有望避免全局同步

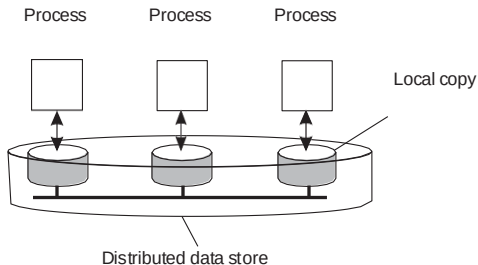
以数据为中心的一致性模型

一致性模型

（分布式）数据存储和进程之间的契约，其中数据存储精确指定了并发情况下读写操作的结果。

基本

数据存储是存储的分布式集合：



持续一致性

我们实际上可以谈论一致性的程度

- ▶ 副本的数值可能不同
- ▶ 副本的相对陈旧性可能不同
- ▶ 执行的更新操作（数量和顺序）可能存在差异

Conit

一致性单元⇒指定要测量一致性的数据单元。

示例: Conit

Replica A

Conit			$d = 558$ // distance $g = 95$ // gas $p = 78$ // price
Operation		Result	
< 5, B>	$g \leftarrow g + 45$	[$g = 45$]	
< 8, A>	$g \leftarrow g + 50$	[$g = 95$]	
< 9, A>	$p \leftarrow p + 78$	[$p = 78$]	
< 10, A>	$d \leftarrow d + 558$	[$d = 558$]	

Vector clock A = (11, 5)
 Order deviation = 3
 Numerical deviation = (2, 482)

Replica B

Conit			$d = 412$ // distance $g = 45$ // gas $p = 70$ // price
Operation		Result	
< 5, B>	$g \leftarrow g + 45$	[$g = 45$]	
< 6, B>	$p \leftarrow p + 70$	[$p = 70$]	
< 7, B>	$d \leftarrow d + 412$	[$d = 412$]	

Vector clock B = (0, 8)
 Order deviation = 1
 Numerical deviation = (3, 686)

Conit (包含变量 g , p , and d)

- 每个副本都有一个矢量时钟: ([known] time @ A, [known] time @ B)
- B 向 A 发送操作 $[(5, B) : g \leftarrow d + 45]$; A 已使此操作永久生效 (无法回滚)

示例: Conit

Replica A

Conit		
d = 558 // distance g = 95 // gas p = 78 // price		
Operation		Result
< 5, B>	g } g + 45	[g = 45]
< 8, A>	g } g + 50	[g = 95]
< 9, A>	p } p + 78	[p = 78]
< 10, A>	d } d + 558	[d = 558]

Vector clock A = (11, 5)
 Order deviation = 3
 Numerical deviation = (2, 482)

Replica B

Conit		
d = 412 // distance g = 45 // gas p = 70 // price		
Operation		Result
< 5, B>	g } g + 45	[g = 45]
< 6, B>	p } p + 70	[p = 70]
< 7, B>	d } d + 412	[d = 412]

Vector clock B = (0, 8)
 Order deviation = 1
 Numerical deviation = (3, 686)

Conit (包含变量 g , p , and d)

- ▶ A 有三项待处理操作 $\Rightarrow$ 订单偏差 = 3
- ▶ A 错过了 B 的两项操作；最大差异为 $70 + 412$ 个单位 $\Rightarrow$ (2, 482)

符号

读写操作

- ▶ $W_i(x)a$: 进程 P_i 将值 a 写入 x
- ▶ $R_i(x)b$: 进程 P_i 从 x 读取值 b
- ▶ 所有数据项最初都具有值 NIL

可能的行为

我们尽可能省略索引并根据时间（ x 轴）绘制：

P1:	<u>W(x)a</u>		
P2:		R(x)NIL	R(x)a

顺序一致性

定义

任何执行的结果都与所有进程的操作按照某种顺序执行的结果相同，并且每个单独进程的操作都按照其程序指定的顺序出现在这个序列中。

(a) 顺序一致的数据存储。 (b) 非顺序一致的数据存

P1:	W(x)a		
P2:	W(x)b		
P3:		R(x)b	R(x)a
P4:		R(x)b	R(x)a

(a)

P1:	W(x)a		
P2:	W(x)b		
P3:		R(x)b	R(x)a
P4:		R(x)a	R(x)b

(b)

示例

三个并发进程（初始值：0）

Process P_1	Process P_2	Process P_3
$x \leftarrow 1;$	$y \leftarrow 1;$	$z \leftarrow 1;$
<code>print (y, z);</code>	<code>print (x, z);</code>	<code>print (x, y);</code>

示例

三个并发进程（初始值：0）

Process P_1	Process P_2	Process P_3
$x \leftarrow 1;$	$y \leftarrow 1;$	$z \leftarrow 1;$
$\text{print}(y, z);$	$\text{print}(x, z);$	$\text{print}(x, y);$

Exam示例执行序列 (signature = [output P_1 output P_2 output P_3])

Execution 1	Execution 2	Execution 3	Execution 4
$P_1: x \leftarrow 1;$ $P_1: \text{print}(y, z);$ $P_2: y \leftarrow 1;$ $P_2: \text{print}(x, z);$ $P_3: z \leftarrow 1;$ $P_3: \text{print}(x, y);$ <i>Prints:</i> 001011 <i>Signature:</i> 001011	$P_1: x \leftarrow 1;$ $P_2: y \leftarrow 1;$ $P_2: \text{print}(x, z);$ $P_1: \text{print}(y, z);$ $P_3: z \leftarrow 1;$ $P_3: \text{print}(x, y);$ <i>Prints:</i> 101011 <i>Signature:</i> 101011	$P_2: y \leftarrow 1;$ $P_3: z \leftarrow 1;$ $P_3: \text{print}(x, y);$ $P_2: \text{print}(x, z);$ $P_1: x \leftarrow 1;$ $P_1: \text{print}(y, z);$ <i>Prints:</i> 010111 <i>Signature:</i> 110101	$P_2: y \leftarrow 1;$ $P_1: x \leftarrow 1;$ $P_3: z \leftarrow 1;$ $P_2: \text{print}(x, z);$ $P_1: \text{print}(y, z);$ $P_3: \text{print}(x, y);$ <i>Prints:</i> 111111 <i>Signature:</i> 111111

因果一致性

定义

所有进程必须以相同的顺序查看具有潜在因果关系的写入。不同进程可能会以不同的顺序查看并发写入。

(a) 违反因果一致性存储。 (b) 因果一致性存储中的正确事件序列

P1: W(x)a			
P2:	R(x)a	W(x)b	
P3:		R(x)b	R(x)a
P4:		R(x)a	R(x)b

(a)

P1: W(x)a			
P2:	W(x)b		
P3:		R(x)b	R(x)a
P4:		R(x)a	R(x)b

(b)

分组操作

条目一致性：定义

- ▶ 对锁的访问是顺序一致的。
- ▶ 在所有先前的写入操作都已完成之前，不允许执行对锁的访问。
- ▶ 在所有先前的锁访问都已完成之前，不允许执行任何数据访问。

分组操作

条目一致性：定义

- ▶ 对锁的访问是顺序一致的。
- ▶ 在所有先前的写入操作都已完成之前，不允许执行对锁的访问。
- ▶ 在所有先前的锁访问都已完成之前，不允许执行任何数据访问。

基本思想

不必关心一系列操作的读写是否立即被其他进程知晓。只希望该系列操作本身的效果被知晓。

分组操作

确保输入一致性的有效事件序列

P1:	L(x) W(x)a L(y) W(y)b U(x) U(y)	
P2:	L(x) R(x)a	R(y) NIL
P3:	L(y) R(y)b	

观察

条目一致性意味着我们需要锁定和解锁数据（隐式或非隐式）。

问题

有什么便捷的方法可以使这种一致性对于程序员来说或多或少透明呢？

以客户为中心的一致性：符号

符号

- ▶ $W_1(x_2)$ 是进程 P_1 执行的写入操作，生成 x 的版本 x_2
- ▶ $W_1(x_i; x_j)$ 表示 P_1 根据前一个版本 x_i 生成版本 x_j
- ▶ $W_1(x_i | x_j)$ 表示 P_1 与版本 x_i 同时生成版本 x_j

Monotonic reads单调读

定义

如果某个进程读取了数据项 x 的值，那么该进程对 x 的任何后续读取操作都将始终返回相同或更新的值。

单个进程 P 在同一数据存储的两个不同本地副本上执行的读取操作。 (a) 单调读取一致的数据存储。 (b) 不提供单调读取的数据存储

L1:	$W_1(x_1)$	$R_1(x_1)$
L2:	$W_2(x_1 x_2)$	$R_1(x_2)$

L1:	$W_1(x_1)$	$R_1(x_1)$
L2:	$W_2(x_1 x_2)$	$R_1(x_2)$

Monotonic reads单调读

示例

自动从不同的服务器读取您的个人日历更新。**Monotonic Reads** 保证用户可以看到所有更新，无论自动读取从哪个服务器进行。

示例

在旅途中阅读（而不是修改）收到的邮件。每次您连接到不同的电子邮件服务器时，该服务器都会从您之前访问的服务器获取（至少）所有更新。

Monotonic writes单调写

定义

某个进程对数据项 x 的写操作在同一进程对 x 进行任何后续写操作之前完成。

- (a) 单调写入一致的数据存储。(b) 不提供单调写入一致性的数据存储。(c) 同样, $WS(x_1|x_2)$ 不具有一致性, 因此 $WS(x_1|x_3)$ 也不具有一致性。(d) 尽管 x_1 显然已经覆盖了 x_2 , 但 $WS(x_1;x_3)$ 仍具有一致性。

L1:	$W_1(x_1)$	
L2:	$W_2(x_1;x_2)$	$W_1(x_2;x_3)$

(a)

L1:	$W_1(x_1)$	
L2:	$W_2(x_1 x_2)$	$W_1(x_1 x_3)$

(b)

L1:	$W_1(x_1)$	
L2:	$W_2(x_1 x_2)$	$W_1(x_2;x_3)$

(c)

L1:	$W_1(x_1)$	
L2:	$W_2(x_1 x_2)$	$W_1(x_1;x_3)$

(d)

Monotonic writes单调写

示例

更新服务器 S2 上的程序，并确保编译和链接所依赖的所有组件也放在 S2 上。

示例

在任何地方都以正确的顺序维护复制文件的版本（将以前的版本传播到安装了最新版本的服务器）。

写后读

定义

进程对数据项 x 的写入操作所产生的影响，将始终被同一进程对 x 的连续读取操作所看到。

(a) 提供“读你所写”一致性的数据存储。

(b) 不提供一致性的数据存储。

L1:	$W_1(x_1)$	
L2:	$W_2(x_1; x_2)$	$R_1(x_2)$

(a)

L1:	$W_1(x_1)$	
L2:	$W_2(x_1 x_2)$	$R_1(x_2)$

(b)

写后读

定义

进程对数据项 x 的写入操作所产生的影响，将始终被同一进程对 x 的连续读取操作所看到。

(a) 提供“读你所写”一致性的数据存储。

(b) 不提供一致性的数据存储。

L1:	$W_1(x_1)$	
L2:	$W_2(x_1; x_2)$	$R_1(x_2)$

(a)

L1:	$W_1(x_1)$	
L2:	$W_2(x_1 x_2)$	$R_1(x_2)$

(b)

示例

更新您的网页并确保您的 **Web** 浏览器显示最新版本而不是其缓存的副本。

读后写

定义

一个进程对数据项 x 执行写操作，紧接着该进程对 x 执行读操作，这个写操作保证发生在与之前读取的 x 值相同或更新的值上。

a) 读后写一致性数据存储。 (b)
不提供读后写一致性的数据存储

示例

仅当您拥有原始帖子时才可以看到对已发布文章的反应（读取“拉入”相应的写入操作）。

L1:	$W_1(x_1)$	$R_2(x_1)$
L2:	$W_3(x_1; x_2)$	$W_2(x_2; x_3)$

(a)

L1:	$W_1(x_1)$	$R_2(x_1)$
L2:	$W_3(x_1 x_2)$	$W_2(x_1 x_3)$

(b)

Replica placement副本放置

本质

. 找出 N 个可能位置中最佳的 K 个位置。

Replica placement副本放置

本质

- . 找出 N 个可能位置中最佳的 K 个位置。
 - ▶ 从 $N-K$ 中选择与客户端的平均距离最小的最佳位置。然后选择下一个最佳服务器。（注意：第一个选定的位置使与所有客户端的平均距离最小。）计算成本高昂。

Replica placement副本放置

本质

. 找出 N 个可能位置中最佳的 K 个位置。

- ▶ 从 $N-K$ 中选择与客户端的平均距离最小的最佳位置。然后选择下一个最佳服务器。（注意：第一个选定的位置使与所有客户端的平均距离最小。）计算成本高昂。
- ▶ 选择第 K 个最大自治系统，并将服务器放置在连接性最好的主机上。计算成本很高。

Replica placement副本放置

本质

. 找出 N 个可能位置中最佳的 K 个位置。

- ▶ 从 $N-K$ 中选择与客户端的平均距离最小的最佳位置。然后选择下一个最佳服务器。（注意：第一个选定的位置使与所有客户端的平均距离最小。）计算成本高昂。
- ▶ 选择第 K 个最大自治系统，并将服务器放置在连接性最好的主机上。计算成本很高。
- ▶ 将节点放置在 d 维几何空间中，其中距离反映延迟。确定密度最高的 K 个区域并在每个区域放置一个服务器。计算成本低。

Content replication内容副本

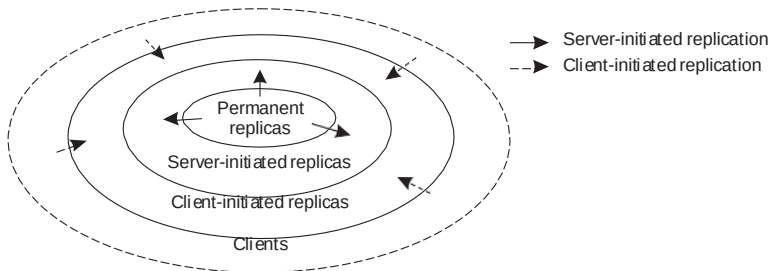
区分不同的进程

进程能够托管对象或数据的副本：

- ▶ **Permanent replicas永久副本：** 进程/机器始终具有副本
- ▶ **Server-initiated replica 服务器启动的副本：** 可以根据数据存储中另一台服务器的请求动态托管副本的进程
- ▶ **Client-initiated replica 客户端启动的副本：** 可以根据客户端（客户端缓存）的请求动态托管副本的进程

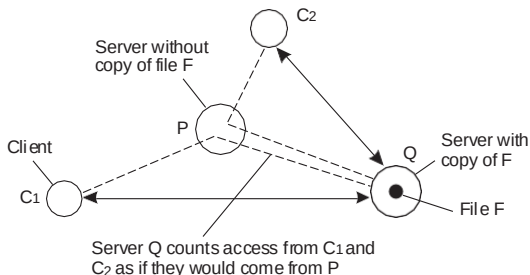
Content replication内容副本

将数据存储的不同类型副本逻辑组织成三个同心环



Server-initiated replicas 服务器启动的副本

统计不同客户端的访问请求



- ▶ 跟踪每个文件的访问次数，通过考虑最接近请求客户端的服务器进行汇总
- ▶ 访问次数低于阈值 $D \Rightarrow$ 删除文件
- ▶ 访问次数超过阈值 $R \Rightarrow$ 复制文件
- ▶ D 和 R 之间的访问次数 $\Rightarrow$ 迁移文件

Content distribution 内容分发

仅考虑客户端-服务器组合

- ▶ 仅传播更新的通知/无效（通常用于缓存）
- ▶ 将数据从一个副本传输到另一个副本（分布式数据库）：被动复制
- ▶ 将更新操作传播到其他副本：主动复制

注意

没有一种方法是最好的，但高度依赖于副本的可用带宽和读写比。

内容分发：客户端/服务器系统

在多客户端、单服务器系统中，基于推送和基于拉取的协议之间的比较

- ▶ 推送更新：服务器发起的方法，无论目标是否要求，都会传播更新。
- ▶ 拉取更新：客户端发起的方法，客户端请求更新。

Issue	Push-based	Pull-based
1:	List of client caches	None
2:	Update (and possibly fetch update)	Poll and update
3:	Immediate (or fetch-update time)	Fetch-update time
<i>1: State at server</i> <i>2: Messages to be exchanged</i> <i>3: Response time at the client</i>		

Content distribution内容分发

观察

我们可以使用租约在拉取和推送之间动态切换：服务器承诺在租约到期之前向客户端推送更新的合同。

使租约到期时间取决于系统的行为（自适应租约）

Content distribution内容分发

观察

我们可以使用租约在拉取和推送之间动态切换：服务器承诺在租约到期之前向客户端推送更新的合同。

使租约到期时间取决于系统的行为（自适应租约）

- **Age-based leases**基于年龄的租约：一个长期没有改变的对象，在不久的将来也不会改变，因此提供长期租约

Content distribution内容分发

观察

我们可以使用租约在拉取和推送之间动态切换：服务器承诺在租约到期之前向客户端推送更新的合同。

使租约到期时间取决于系统的行为（自适应租约）

- **Age-based leases**基于年龄的租约：一个长期没有改变的对象，在不久的将来也不会改变，因此提供长期租约
- **Renewal-frequency based leases**基于续订频率的租约：客户端请求特定对象的频率越高，该客户端（该对象）的到期时间就越长

Content distribution内容分发

观察

我们可以使用租约在拉取和推送之间动态切换：服务器承诺在租约到期之前向客户端推送更新的合同。

使租约到期时间取决于系统的行为（自适应租约）

- **Age-based leases**基于年龄的租约：一个长期没有改变的对象，在不久的将来也不会改变，因此提供长期租约
- **Renewal-frequency based leases**基于续订频率的租约：客户端请求特定对象的频率越高，该客户端（该对象）的到期时间就越长
- **State-based leases**基于状态的租约：服务器负载越大，租约到期时间越短

持续一致性：数值偏差

Principal operation基本操作

- ▶ 每个服务器 S_i 有一个日志 L_i .
- ▶ 考虑数据项 x , $val(W)$ 表示写入操作 W 后其值的数值变化。
假设

$$\forall W : val(W) > 0$$

- ▶ W 最初被转发到 N 个副本中的一个, 表示为 $origin(W)$. $TW[i, j]$ 是源自 S_j 并由服务器 S_i 执行的写入:

$$TW[i, j] = \sum \{val(W) \mid origin(W) = S_j \ \& \ W \in L_i\}$$

持续一致性：数值偏差

注意

x 的实际值 $v(t)$:

$$v(t) = v_{init} + \sum_{k=1}^N TW[k, k]$$

服务器 S_i 上 x 的值 v_i :

$$v_i = v_{init} + \sum_{k=1}^N TW[i, k]$$

持续一致性：数值偏差

问题

我们需要确保对于每个服务器 S_i , $v(t) - v_i < \delta_i$ 。

持续一致性：数值偏差

问题

我们需要确保对于每个服务器 S_i , $v(t) - v_i < \delta_i$ 。

方法

让每个服务器 S_k 维护一个视图 $TW_k [i, j]$, 即它认为 $TW [i, j]$ 的值。当更新被传播时, 这个信息可以被传播。

持续一致性：数值偏差

问题

我们需要确保对于每个服务器 S_i , $v(t) - v_i < \delta_i$ 。

方法

让每个服务器 S_k 维护一个视图 $TW_k [i, j]$, 即它认为 $TW [i, j]$ 的值。当更新被传播时, 这个信息可以被传播。

注意

$$0 \leq TW_k [i, j] \leq TW [i, j] \leq TW [j, j]$$

持续一致性：数值偏差

解决方法

当 S_k 发现 $TW_k[i, k]$ 离 $TW[k, k]$ 太远时，它会将其日志中的操作发送给 S_i ，特别是当

$$TW[k, k] - TW_k[i, k] > \delta_i / (N - 1)$$

持续一致性：数值偏差

解决方法

当 S_k 发现 $TW_k[i, k]$ 离 $TW[k, k]$ 太远时，它会将其日志中的操作发送给 S_i ，特别是当

$$TW[k, k] - TW_k[i, k] > \delta_i / (N - 1)$$

Question问题

我们在多大程度上是悲观的： $\delta_i / (N - 1)$ 从何而来？

持续一致性：数值偏差

解决方法

当 S_k 发现 $TW_k[i, k]$ 离 $TW[k, k]$ 太远时，它会将其日志中的操作发送给 S_i ，特别是当

$$TW[k, k] - TW_k[i, k] > \delta_i / (N - 1)$$

Question问题

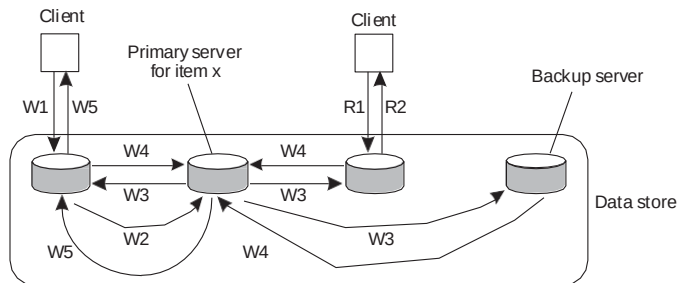
我们在多大程度上是悲观的： $\delta_i / (N - 1)$ 从何而来？

注意

过期时间可以通过类似方式实现，本质上是通过跟踪 S_i 最后看到的内容

Primary-based protocols 基于主协议

主备份协议

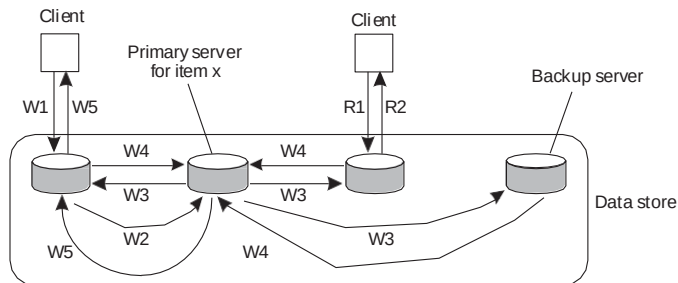


W1. Write request
W2. Forward request to primary
W3. Tell backups to update
W4. Acknowledge update
W5. Acknowledge write completed

R1. Read request
R2. Response to read

Primary-based protocols 基于主协议

主备份协议



W1. Write request
W2. Forward request to primary
W3. Tell backups to update
W4. Acknowledge update
W5. Acknowledge write completed

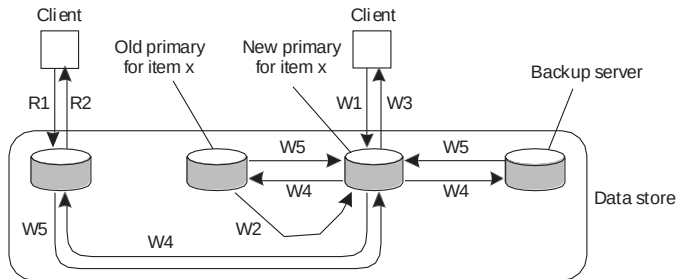
R1. Read request
R2. Response to read

主备份协议示例

传统上应用于需要高容错能力的分布式数据库和文件系统。副本通常放置在同一个 LAN 上。

Primary-based protocols 基于主协议

具有本地写入的主备份协议

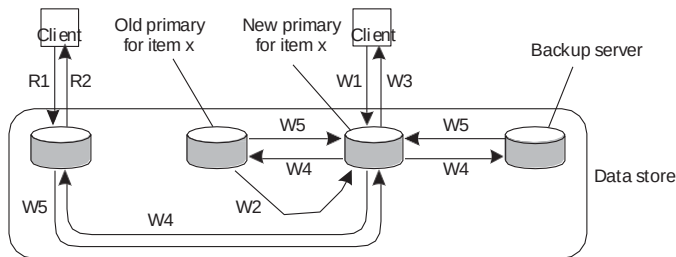


W1. Write request
W2. Move item x to new primary
W3. Acknowledge write completed
W4. Tell backups to update
W5. Acknowledge update

R1. Read request
R2. Response to read

Primary-based protocols

具有本地写入的主备份协议



W1. Write request
W2. Move item x to new primary
W3. Acknowledge write completed
W4. Tell backups to update
W5. Acknowledge update

R1. Read request
R2. Response to read

具有本地写入的主备份协议示例

断开连接模式下的移动计算（断开连接前将所有相关文件发送给用户，稍后再更新）。